

Actividad Final DatAI

Serene: base de datos para automatizar reservas en salud mental

Plataforma elegida: Supabase PostgreSQL

Repositorio: <https://github.com/sebasgc4408/SereneBDDfinal>

Proyecto en Supabase: <https://supabase.com/dashboard/project/kkrugcnfqftprdifibup>

Integrantes: Sebastián Guerrero y Mathias Velez

1. Introduccion

Para esta actividad trabajamos sobre **Serene**, una idea de aplicacion orientada a psicologos independientes y pequenos consultorios de salud mental. El objetivo de Serene es reducir la carga administrativa que aparece alrededor de las citas: coordinar horarios, revisar disponibilidad, evitar cruces con el calendario, registrar pacientes, enviar recordatorios y hacer seguimiento despues de una sesion.

Aunque la aplicacion original tenia una interfaz y un backend basado en Convex, para la entrega final del curso decidimos aterrizar el proyecto en una plataforma del catalogo visto en clase. Por eso montamos una version relacional de la base de datos en **Supabase PostgreSQL**. La meta fue que la base quedara desplegada, con datos cargados y con consultas capaces de responder preguntas reales del negocio.

En pocas palabras, el proyecto busca mostrar como una base de datos puede ayudar a que un psicologo entienda mejor su agenda: que tanto se usa su disponibilidad, que canales generan mas reservas, cuantos pacientes no asisten, que citas requieren seguimiento y que indicadores operativos se pueden mejorar.

2. Problema que resuelve

En una consulta psicologica pequena, muchas tareas importantes no son clinicas, sino administrativas. Por ejemplo, confirmar citas, revisar si hay espacio disponible, atender solicitudes por diferentes canales, evitar choques con Google Calendar y recordar a los pacientes sus sesiones. Cuando estas tareas se manejan manualmente, es facil que aparezcan errores, cancelaciones o tiempos muertos en la agenda.

Serene plantea una solucion a ese problema. La plataforma centraliza la informacion de psicologos, pacientes, disponibilidad, citas, solicitudes de reserva, integraciones y seguimientos. Con esa informacion organizada en una base de datos, el consultorio puede pasar de solo "guardar citas" a tomar decisiones: detectar horarios con alta demanda, medir la ocupacion semanal, revisar no-shows y entender que canal convierte mejor en citas reales.

Este caso nos parecia adecuado para la actividad porque tiene datos transaccionales claros, relaciones entre entidades y consultas con sentido de negocio. No se trata solo de crear tablas, sino de construir una base viva y consultable.

3. Por que elegimos Supabase

Elegimos **Supabase PostgreSQL** porque Serene es un caso principalmente relacional. Un psicologo tiene muchos pacientes, muchas citas, varias reglas de disponibilidad y diferentes eventos de calendario. A su vez, una cita puede estar asociada a un paciente, a una solicitud de reserva y a un seguimiento posterior. Este tipo de estructura encaja muy bien con PostgreSQL, porque permite usar llaves primarias, llaves foraneas, restricciones, indices y consultas SQL avanzadas.

Supabase tambien fue practico para el alcance de la actividad. Permite crear una base en la nube desde el navegador, ejecutar scripts SQL en el editor, revisar tablas en el panel y tomar capturas facilmente para la entrega. Ademias, al estar basado en PostgreSQL, permite hacer consultas con `joins`, agregaciones, CTEs y funciones de ventana.

Comparamos Supabase con otras dos plataformas del catalogo:

- **MongoDB Atlas:** es una buena opcion cuando los datos son muy flexibles o documentales. Sin embargo, para Serene era importante conservar relaciones fuertes entre psicologos, pacientes, citas y solicitudes. Por eso un modelo relacional era mas conveniente.
- **BigQuery:** es excelente para analitica sobre grandes volumenes de datos. Aun asi, Serene necesitaba una base transaccional para registrar reservas y disponibilidad, no solo un almacen analitico. Para este caso, BigQuery hubiera sido mas grande de lo necesario.

Por esas razones, Supabase fue una opcion equilibrada: facil de desplegar, permitida por el curso, relacional y suficientemente potente para responder las preguntas del proyecto.

4. Modelo de datos

El modelo de datos quedo organizado alrededor de ocho tablas principales. La idea fue separar con claridad las partes del negocio: por un lado los psicologos y pacientes; por otro, la disponibilidad, las citas, las solicitudes de reserva, los eventos de calendario, las integraciones y los seguimientos.

Las tablas principales son:

- `psychologists`: almacena los psicologos registrados, su correo, nombre, enlace publico, zona horaria y estado general de integracion.
- `patients`: guarda los pacientes asociados a cada psicologo. Incluye datos de contacto, notas y un puntaje NPS opcional.
- `appointments`: registra las citas. Esta es una de las tablas centrales, porque permite saber si una cita fue confirmada, completada, cancelada o marcada como no-show.

- `availability_rules` : define los días, horas, duracion de sesion, descansos y modalidad de atencion de cada psicologo.
- `calendar_events` : representa bloques ocupados importados desde Google Calendar. Sirve para evitar cruces con citas nuevas.
- `booking_requests` : guarda las solicitudes de reserva antes de convertirse en cita. Permite analizar canales como web, WhatsApp o manual.
- `follow_ups` : registra acciones posteriores a una cita, como seguimientos clinicos o contacto despues de una inasistencia.
- `integrations` : resume el estado de conexiones externas, como Google Calendar, WhatsApp, email y Twilio.

El esquema usa llaves primarias tipo `uuid` y llaves foraneas para conectar las entidades. Por ejemplo, `patients`, `appointments`, `availability_rules`, `booking_requests`, `calendar_events`, `follow_ups` e `integrations` se relacionan con `psychologists`. Además, `appointments`, `booking_requests` y `follow_ups` pueden relacionarse con `patients`. Esta estructura hace posible consultar la operacion completa de un psicologo desde varios angulos.



En esa imagen se deben ver las tablas `psychologists`, `patients`, `appointments`, `availability_rules`, `calendar_events`, `booking_requests`, `follow_ups` e `integrations`, junto con sus relaciones.

El modelo permite responder preguntas tanto operativas como analíticas. Por ejemplo, desde `appointments` se calculan tasas de finalización o no-show; desde `availability_rules` y `appointments` se mide la ocupación semanal; y desde `booking_requests` se analiza la conversión por canal.

5. Base desplegada y datos cargados

La base fue creada en Supabase y cargada con datos sintéticos. Usamos datos ficticios porque el tema de salud mental puede involucrar información sensible, y no era necesario usar datos reales para demostrar el modelo.

Los datos cargados fueron:

- 8 psicólogos.
- 240 pacientes.
- 42 reglas de disponibilidad.
- 160 eventos de calendario.
- 900 citas.
- 320 solicitudes de reserva.
- 180 seguimientos.
- 8 registros de integración.

Estos datos permiten simular el funcionamiento de Serene durante varias semanas y ejecutar consultas de negocio con resultados variados.

table_name	total_rows
appointments	900
availability_rules	42
booking_requests	320
calendar_events	160
follow_ups	180
integrations	8
patients	240
psychologists	8

6. Consultas de negocio

El archivo `02_business_queries.sql` contiene las consultas usadas para analizar la base. A continuacion se resumen las mas importantes.

6.1 KPIs por psicologo

Esta consulta compara a los psicologos segun total de citas, citas completadas, cancelaciones, no-shows, tasa de finalizacion y tasa de no-show. Es util porque permite ver rapidamente que profesionales tienen una agenda mas estable y cuales pueden necesitar apoyo operativo.

Interpretacion: si un psicologo tiene muchas citas pero una tasa alta de no-show, Serene podria reforzar recordatorios o revisar los horarios mas conflictivos.

```
select
  full_name,
  total_appointments,
  completed_appointments,
  cancelled_appointments,
  no_show_appointments,
  completion_rate,
  no_show_rate
from v_psychologist_kpis
order by total_appointments desc, completion_rate desc;
```

6.2 Ocupacion semanal de agenda

Esta consulta compara los minutos disponibles configurados por cada psicologo contra los minutos efectivamente reservados en una semana. Sirve para medir que tanto se esta usando la capacidad real de atencion.

Interpretacion: una ocupacion baja puede indicar que hay demasiados espacios disponibles, poca demanda o problemas de visibilidad del enlace de reserva.

```

with params as (
  select date '2026-05-18' as week_start
),
available_minutes as (
  select
    p.id as psychologist_id,
    p.full_name,
    sum(extract(epoch from (ar.end_time - ar.start_time)) / 60) as available_minutes
  from psychologists p
  join availability_rules ar on ar.psychologist_id = p.id
  where ar.is_active
  group by p.id, p.full_name
),
booked_minutes as (
  select
    a.psychologist_id,
    sum(extract(epoch from (a.end_at - a.start_at)) / 60) as booked_minutes
  from appointments a
  cross join params pr
  where a.start_at >= pr.week_start
    and a.start_at < pr.week_start + interval '7 days'
    and a.status in ('confirmed', 'completed', 'no_show')
  group by a.psychologist_id
)
select
  am.full_name,
  am.available_minutes,
  coalesce(bm.booked_minutes, 0) as booked_minutes,
  round((100.0 * coalesce(bm.booked_minutes, 0) / nullif(am.available_minutes, 0))::numeric, 2) as occupancy_rate
from available_minutes am
left join booked_minutes bm on bm.psychologist_id = am.psychologist_id
order by occupancy_rate desc;

```

6.3 Horarios mas solicitados por canal

Esta consulta muestra que horarios tienen mas solicitudes y desde que canal llegan. Esto ayuda a entender los momentos de mayor demanda y a ajustar disponibilidad o comunicacion.

Interpretacion: si ciertos horarios concentran solicitudes aprobadas, el psicologo podria abrir mas espacios en esos bloques.

```

select
  requested_time,
  channel,
  count(*) as total_requests,
  count(*) filter (where status = 'approved') as approved_requests,
  round((100.0 * count(*) filter (where status = 'approved') / nullif(count(*), 0))::numeric, 2) as approval_rate
from booking_requests
group by requested_time, channel
order by total_requests desc, approval_rate desc
limit 15;

```

6.4 Conversion por canal

Esta consulta analiza el embudo desde solicitud hasta cita aprobada, agrupando por canal. Permite comparar web, WhatsApp y canal manual.

Interpretacion: si un canal tiene alta conversion, vale la pena fortalecerlo. Si otro tiene muchas solicitudes pero pocas citas aprobadas, puede haber friccion en el proceso.

```
with channel_funnel as (  
  select  
    channel,  
    count(*) as total_requests,  
    count(*) filter (where status = 'approved') as approved_requests,  
    count(*) filter (where appointment_id is not null) as linked_appointments  
  from booking_requests  
  group by channel  
)  
select  
  channel,  
  total_requests,  
  approved_requests,  
  linked_appointments,  
  round((100.0 * approved_requests / nullif(total_requests, 0))::numeric, 2) as approval_rate,  
  round((100.0 * linked_appointments / nullif(total_requests, 0))::numeric, 2) as appointment_conversion_rate  
from channel_funnel  
order by appointment_conversion_rate desc;
```

6.5 Ranking de riesgo operativo

Esta consulta usa una funcion de ventana (`dense_rank`) para ordenar a los psicologos segun una tasa de riesgo basada en cancelaciones y no-shows.

Interpretacion: el ranking permite priorizar intervenciones. Por ejemplo, mejorar recordatorios, revisar horarios o hacer seguimiento a pacientes que faltan con frecuencia.

```
with risk_metrics as (  
  select  
    p.id,  
    p.full_name,  
    count(a.id) as total_appointments,  
    count(*) filter (where a.status = 'cancelled') as cancellations,  
    count(*) filter (where a.status = 'no_show' or a.no_show) as no_shows,  
    (  
      count(*) filter (where a.status = 'cancelled')  
      + count(*) filter (where a.status = 'no_show' or a.no_show)  
    )::numeric / nullif(count(a.id), 0) as risk_rate  
  from psychologists p  
  left join appointments a on a.psychologist_id = p.id  
  group by p.id, p.full_name  
)  
select  
  dense_rank() over (order by risk_rate desc nulls last) as risk_rank,  
  full_name,  
  total_appointments,  
  cancellations,  
  no_shows,  
  round((100.0 * risk_rate)::numeric, 2) as risk_rate_percent  
from risk_metrics  
order by risk_rank, full_name;
```

6.6 Seguimientos pendientes

Esta consulta muestra tareas de seguimiento despues de citas completadas o no-shows. Incluye una prioridad calculada para identificar que acciones requieren atencion mas pronto.

Interpretacion: ayuda a que el consultorio no pierda continuidad con los pacientes y mantenga un flujo de atencion mas ordenado.


```
select
  p.full_name as psychologist_name,
  pa.name as patient_name,
  a.status as appointment_status,
  f.follow_up_at,
  f.notes,
  case
    when f.follow_up_at < '2026-06-01'::timestampz then 'urgent'
    when f.follow_up_at < '2026-06-15'::timestampz then 'soon'
    else 'scheduled'
  end as priority
from follow_ups f
join psychologists p on p.id = f.psychologist_id
left join patients pa on pa.id = f.patient_id
join appointments a on a.id = f.appointment_id
order by f.follow_up_at asc
limit 30;
```

7. Que fue practico y que fue dificil

Lo mas practico fue trabajar con Supabase desde el navegador. Crear el proyecto, ejecutar SQL y revisar los resultados fue directo. Tambien ayudo que PostgreSQL tenga una sintaxis conocida y permita construir consultas bastante expresivas sin herramientas adicionales.

La parte mas dificil fue traducir el modelo original de la aplicacion a un diseno relacional. En Convex, varias entidades estaban pensadas como documentos y funciones. En PostgreSQL fue necesario separar mejor las responsabilidades: pacientes por un lado, citas por otro, disponibilidad, solicitudes e integraciones en tablas independientes. Ese proceso obligo a pensar con mas cuidado las relaciones, las restricciones y los tipos de datos.

Otro reto fue preparar datos sinteticos que fueran suficientemente realistas. No bastaba con insertar filas vacias; los datos debian permitir preguntas utiles, como conversion, ocupacion y no-show.

8. Uso de IA generativa

Se uso IA generativa como apoyo durante el desarrollo del trabajo. La IA ayudo a revisar el repositorio original, identificar las entidades principales, proponer el modelo relacional, escribir los scripts SQL, generar datos sinteticos y redactar una primera version del informe.

El uso de IA no reemplazo la toma de decisiones del grupo. La eleccion de Supabase, la validacion en la plataforma, las capturas y la interpretacion del caso se hicieron de acuerdo con los requisitos de la actividad.

9. Conclusiones

Este proyecto muestra como una base de datos puede convertirse en una herramienta de gestion para un problema real. En el caso de Serene, la base no solo guarda citas, sino que ayuda a entender la operacion de una consulta psicologica: ocupacion, demanda, conversion, seguimiento y riesgo operativo.

Supabase PostgreSQL fue una buena eleccion porque permitio desplegar la base rapidamente y conservar un modelo relacional claro. Ademas, las consultas SQL permitieron responder preguntas de negocio sin depender de una interfaz grafica compleja.

Como resultado final, la base quedo desplegada, cargada con datos sinteticos y lista para ser consultada durante la revision. El ejercicio tambien ayudo a conectar varios temas del curso: modelado de datos, integridad referencial, consultas analiticas, despliegue en la nube y criterio para elegir una plataforma segun el problema.